

Mathematics for Machine Learning — Probability and Statistics

This guide builds the probability and statistics vocabulary used in machine learning, deep learning, NLP, computer vision, and generative AI. Examples use Python 3 with NumPy.

Probability Basics

1. Overview

Probability quantifies uncertainty on a scale from 0 to 1. ML uses it to model noisy data, predictions, risks, and model confidence—for example, a classifier may assign 0.92 probability to “spam.”

2. Intuition

Think of probability as the long-run fraction of an outcome. If a fair coin is tossed many times, heads occurs in roughly half the tosses, so $P(\text{heads})=0.5$.

3. Prerequisites

Arithmetic, fractions, sets, basic Python, and the notions of dataset and feature are sufficient.

4. Core Concepts

- **Sample space (Ω)**: every possible outcome; for one die, $\{1, \dots, 6\}$. It defines what can happen. Interview angle: distinguish an outcome from an event.
- **Event (A)**: a set of outcomes; “even die roll” is $\{2, 4, 6\}$. Its probability is the probability mass in that set.
- **Axioms**: $0 \leq P(A) \leq 1$, $P(\Omega)=1$, and disjoint events add: $P(A \cup B)=P(A)+P(B)$.
- **Complement and union**: $P(A^c)=1-P(A)$ and $P(A \cup B)=P(A)+P(B)-P(A \cap B)$. Avoid double-counting overlap.

5. Algorithm / Working Process

1. Define outcomes and the event of interest.
2. Choose a model (equally likely outcomes, historical frequency, or learned distribution).
3. Sum probabilities of event outcomes.
4. Check that all outcome probabilities total one.

6. Mathematical Foundation

For equally likely outcomes, $P(A)=|A|/|\Omega|$. For a discrete space, $P(A)=\sum_{x \in A} P(X=x)$. In ML, probabilities are non-negative normalized scores, commonly produced by softmax.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(7)
rolls = rng.integers(1, 7, size=100_000)
p_even = np.mean(rolls % 2 == 0)
print(p_even) # close to 0.5
```

8. Code Explanation

`integers` simulates die outcomes; the Boolean expression marks even values, and NumPy treats `True` as 1 when `mean` is computed—an empirical probability.

9. Training / Evaluation

No training is required. With sampled data, more observations reduce sampling error. Validate probabilities by checking normalization and by comparing predicted frequencies with observed frequencies (calibration).

10. Complexity and Cost

Counting n samples costs $O(n)$ time and $O(n)$ memory if stored; streaming counts need $O(1)$ memory. No GPU is needed.

11. Common Use Cases

Click-through-rate estimation, uncertainty-aware classifiers, A/B testing, risk scoring, Monte Carlo simulation, and language-model token probabilities.

12. Common Mistakes

Treating probability as certainty; adding non-disjoint event probabilities; assuming outcomes are equally likely without evidence; using training-set frequency as an unbiased production estimate.

13. Edge Cases / Limitations

Rare events need huge samples; past frequency may not represent a changing population; a probability model may be miscalibrated even when accuracy is high.

14. Variations

- **Frequentist probability** uses repeated-sampling frequency; central for classical estimation.
- **Bayesian probability** represents degree of belief and updates with evidence; vital for uncertainty modeling.
- **Empirical probability** estimates from a finite dataset; essential in projects and placements.

15. Related Topics

Conditional probability restricts the sample space; random variables attach numbers to outcomes; distributions specify probabilities across values; likelihood connects probabilities to parameter learning.

16. Interview Questions

1. **What range can probability take?** $[0, 1]$.
2. **What is a sample space?** The complete set of possible outcomes.
3. **Can two events both have probability 0.8?** Yes; they need not be disjoint.
4. **$P(A \cup B)$?** $P(A) + P(B) - P(A \cap B)$.
5. **What is the complement rule?** $P(A^c) = 1 - P(A)$.
6. **Why must probabilities sum to one?** Exactly one outcome in an exhaustive discrete sample space occurs.
7. **Probability versus odds?** Odds are $p/(1-p)$; probability is p .
8. **Why are softmax outputs probabilities?** They are non-negative and sum to one.
9. **Does 0.7 mean 70% certainty for one item?** It is a model's probabilistic claim; calibration determines whether it is trustworthy.
10. **How estimate an unknown probability?** Count event occurrences divided by trials, with uncertainty intervals.

17. Practice Tasks

Simulate coins and dice; estimate $P(\text{sum}=7)$ for two dice; verify union rules; plot empirical probabilities as sample size grows; assess a classifier's calibration curve.

18. Project Ideas

- **A/B-test simulator** — simulate conversion probability and confidence intervals; NumPy/Matplotlib; synthetic data; demonstrates experimentation.
- **Weather-risk dashboard** — estimate rain-event frequencies; Pandas; weather history; demonstrates data cleaning and uncertainty.
- **Model calibration audit** — compare confidence and accuracy; scikit-learn; CIFAR or spam predictions; strong ML-engineering signal.

19. Quick Revision

Key idea: quantify uncertainty. Formula: $P(A) = |A| / |\Omega|$ only for equally likely outcomes. Use for uncertainty and frequency estimates. Trap: double-counting intersections. One-liner: probability is normalized belief or long-run frequency.

20. Final Cheat Sheet

Item	Summary
Definition	Number in $[0, 1]$ assigned to an event
Input/output	Outcomes $\rightarrow$ event likelihood
Main steps	define space, model, aggregate
Metrics	calibration, log loss where predictions are used
Pros/cons	interpretable uncertainty / depends on model assumptions
Best uses	experiments, classifiers, simulations

Conditional Probability

1. Overview

Conditional probability measures the chance of event **A** after knowing event **B** occurred. It is central to diagnosis, feature dependencies, Naive Bayes, and probabilistic inference.

2. Intuition

If 40 of 100 students are women and 10 are both women and ML interns, then among women the intern probability is $10/40$, not $10/100$: knowledge of **B** narrows the relevant population.

3. Prerequisites

Probability basics, set intersection, fractions, and contingency tables.

4. Core Concepts

- **Definition:** $P(A|B) = P(A \cap B) / P(B)$ for $P(B) > 0$; it renormalizes within **B**.
- **Multiplication rule:** $P(A \cap B) = P(A|B)P(B)$; used to factor joint probabilities.
- **Independence:** **A** and **B** are independent iff $P(A|B) = P(A)$ (when defined), equivalently $P(A \cap B) = P(A)P(B)$.
- **Chain rule:** $P(x_1, \dots, x_n) = \prod_i P(x_i | x_1, \dots, x_{i-1})$; this drives autoregressive language models.

5. Algorithm / Working Process

Filter to observations satisfying **B**, count those also satisfying **A**, then divide by the number satisfying **B**. In a distribution, calculate joint mass and normalize by marginal mass.

6. Mathematical Foundation

The denominator $P(B)$ is evidence probability. For partitions B_i , total probability gives $P(A) = \sum_i P(A|B_i)P(B_i)$. Conditional density uses $p(x|y) = p(x, y) / p(y)$.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(0)
study = rng.random(10_000) < 0.60
# A deliberately dependent outcome: study raises pass chance.
passed = rng.random(10_000) < np.where(study, 0.85, 0.45)
print("P(pass | study) =", passed[study].mean())
print("P(pass) =", passed.mean())
```

8. Code Explanation

Boolean indexing constructs the conditional population (study). Taking $\text{mean of passed}[\text{study}]$ estimates $P(\text{pass}|\text{study})$.

9. Training / Evaluation

Estimate conditionals on training data but evaluate on a held-out representative split. Sparse conditions cause high variance; smooth, pool categories, or use a model rather than raw counts.

10. Complexity and Cost

Filtering/counting is $O(n)$ time. A dense conditional probability table grows exponentially with number of conditioning variables, causing the curse of dimensionality.

11. Common Use Cases

Medical tests, fraud given transaction features, Naive Bayes, Bayesian networks, recommender systems, and next-token prediction.

12. Common Mistakes

Confusing $P(A|B)$ with $P(B|A)$; dividing by zero; calling correlated variables causal; estimating tiny conditional groups from too few observations.

13. Edge Cases / Limitations

$P(A|B)$ is undefined if $P(B)=0$; conditioning on a collider can create spurious association; high-dimensional conditioning is data hungry.

14. Variations

- **Conditional density** for continuous features, used in generative models.
- **Conditional independence** simplifies models (Naive Bayes); placement-critical.
- **Regular conditional distributions** generalize conditioning rigorously; research-level.

15. Related Topics

Bayes' theorem reverses a conditional; joint distributions represent both variables; graphical models encode many conditional independencies; attention models conditionalize token distributions on context.

16. Interview Questions

1. **Define $P(A|B)$.** $P(A \cap B) / P(B)$ when $P(B) > 0$.
2. **Is $P(A|B) = P(B|A)$?** Usually no.
3. **Test independence?** Check whether $P(A|B) = P(A)$ or joint equals product.
4. **What does conditioning do?** Restricts and renormalizes the sample space.
5. **Joint from conditional?** $P(A, B) = P(A|B)P(B)$.
6. **Why does Naive Bayes help?** It factors a difficult conditional using an independence assumption.
7. **Why can conditional probabilities be misleading?** Confounding, selection bias, and small samples.
8. **What if evidence has zero probability?** The elementary formula is undefined.
9. **State the chain rule.** Product of sequential conditionals.

10. **LLM connection?** It predicts $P(\text{next token} \mid \text{prior tokens})$.

17. Practice Tasks

Build a contingency table; compute conditionals from Titanic data; test whether two features are independent; implement a chain-rule sequence likelihood.

18. Project Ideas

- **Loan-risk explorer** — conditional default rates by features; Pandas; Lending Club-like data; interpretable analytics.
- **Medical-test calculator** — sensitivity/specificity conditionals; Streamlit; synthetic prevalence scenarios; strong interview demo.
- **Next-word frequency model** — bigram conditionals; Python/NumPy; public text corpus; bridges to NLP.

19. Quick Revision

$P(A|B)$ means “A among B.” Formula: joint divided by evidence. Use when new information changes uncertainty. Trap: reversing the condition. One-liner: conditioning changes the reference population.

20. Final Cheat Sheet

Item	Summary
Definition	Probability of A after B is known
Input/output	joint and evidence $\rightarrow$ conditional
Key equation	$P(A$
Cost	tables explode with dimensions
Best uses	diagnosis, sequences, Bayesian models

Bayes' Theorem

1. Overview

Bayes' theorem updates a prior belief using observed evidence. It powers spam filters, medical diagnosis, Bayesian optimization, uncertainty-aware ML, and Bayesian neural-network methods.

2. Intuition

A positive rare-disease test is not automatically strong evidence: Bayes combines test accuracy with disease prevalence. Base rates often matter more than intuition expects.

3. Prerequisites

Probability basics, conditional probability, and algebra.

4. Core Concepts

- **Prior** $P(H)$: belief before data; captures prevalence or domain knowledge.
- **Likelihood** $P(E|H)$: how compatible evidence is with a hypothesis; it scores hypotheses.
- **Evidence** $P(E)$: probability of observing data under all hypotheses; it normalizes.
- **Posterior** $P(H|E)$: updated belief. Interview angle: distinguish likelihood from posterior.

5. Algorithm / Working Process

1. List hypotheses and assign priors.
2. Specify likelihood of evidence under each.
3. Compute evidence by total probability.
4. Multiply prior by likelihood and normalize to get posterior.
5. Use the posterior for a decision or update it again with new data.

6. Mathematical Foundation

$P(H|E) = P(E|H)P(H)/P(E)$, where $P(E) = \sum_h P(E|h)P(h)$. In parameter estimation, $p(\theta|D) \propto p(D|\theta)p(\theta)$. MAP maximizes posterior; MLE maximizes likelihood.

7. Practical Implementation

```
# Disease prevalence=1%, sensitivity=90%, false-positive rate=5%
prior, sensitivity, fpr = 0.01, 0.90, 0.05
p_positive = sensitivity * prior + fpr * (1 - prior)
posterior = sensitivity * prior / p_positive
print(f"P(disease | positive) = {posterior:.1%}")
```

8. Code Explanation

The denominator adds positive-test probability from diseased and non-diseased people. The numerator is the joint probability of disease and a positive test.

9. Training / Evaluation

Bayesian models fit likelihood parameters and possibly priors. Evaluate posterior predictive performance, calibration, log likelihood, and decision cost—not only accuracy. Use validation data to select priors/hyperparameters carefully.

10. Complexity and Cost

Finite hypothesis tables are cheap. Exact posterior inference can be exponential or intractable in complex models; MCMC and variational inference trade accuracy for compute.

11. Common Use Cases

Spam classification, disease screening, root-cause diagnosis, A/B testing, sensor fusion, active learning, and retrieval reranking.

12. Common Mistakes

Ignoring base rates; swapping $P(E|H)$ and $P(H|E)$; using an unjustified prior; treating posterior probability as causal proof; double-counting correlated evidence.

13. Edge Cases / Limitations

Bad priors or misspecified likelihoods yield misleading posteriors. Rare evidence may cause numerical underflow; compute in log space for real systems.

14. Variations

- **Naive Bayes** assumes conditionally independent features; highly placement-relevant.
- **MAP estimation** adds a prior to MLE; useful with little data.
- **Bayesian networks** represent many variables graphically; useful in diagnosis.
- **Variational Bayes/MCMC** approximate difficult posteriors; research and generative AI relevant.

15. Related Topics

Conditional probability provides the formula; MLE omits the prior; Bayesian inference returns distributions over parameters; calibration evaluates posterior-like model scores.

16. Interview Questions

1. **State Bayes' theorem.** $P(H|E) = P(E|H)P(H)/P(E)$.
2. **What is a prior?** Belief about a hypothesis before current evidence.
3. **Likelihood versus posterior?** Likelihood varies evidence given parameter; posterior varies parameter given evidence.
4. **Why is evidence needed?** To normalize posterior probabilities to sum to one.
5. **What is base-rate fallacy?** Ignoring $P(H)$.
6. **MAP versus MLE?** MAP maximizes likelihood times prior; MLE uses likelihood only.
7. **When does MAP approach MLE?** With abundant data or a weak/uniform prior.
8. **Can priors be data-driven?** Yes, but avoid using test data and double-counting evidence.
9. **Why log probabilities?** Products of small values underflow; logs turn products into sums.
10. **Naive Bayes assumption?** Features are independent conditional on class.

17. Practice Tasks

Compute posterior disease risk under several prevalences; code a categorical Naive Bayes classifier; compare MLE and MAP with small data; plot posterior after repeated coin tosses.

18. Project Ideas

- **Spam classifier** — Multinomial Naive Bayes; scikit-learn; SMS Spam data; classic placement project.
- **Bayesian A/B tester** — beta-binomial posteriors; NumPy/Streamlit; conversion logs; decision-making focus.
- **Failure diagnoser** — Bayesian network prototype; pgmpy; synthetic equipment data; demonstrates reasoning under uncertainty.

19. Quick Revision

Posterior = likelihood $\times$ prior $\div$ evidence. Use to update beliefs from observations. Trap: confusing reverse conditionals. One-liner: Bayes converts “evidence if hypothesis” into “hypothesis given evidence.”

20. Final Cheat Sheet

Item	Summary
Input/output	prior + likelihood + evidence $\rightarrow$ posterior
Main equation	$p(\theta)$
Metrics	log likelihood, calibration, expected decision cost
Pros/cons	uses prior knowledge / inference may be costly
Best uses	small data, diagnosis, uncertainty

Random Variables

1. Overview

A random variable (RV) maps uncertain outcomes to numbers. It makes probability usable for ML features, labels, losses, and noise models.

2. Intuition

For a die, the outcome is a face; X can be the displayed number. For two dice, X can instead be their sum. One experiment can define many RVs.

3. Prerequisites

Probability basics, functions, and elementary algebra.

4. Core Concepts

- **Discrete RV** has countable values, e.g., number of clicks; described by a PMF $P(X=x)$.
- **Continuous RV** takes values on intervals, e.g., height; described by PDF $f(x)$, not point probabilities.
- **CDF** $F(x)=P(X\leq x)$ works for both types; interview angle: it is nondecreasing and ends at 1.
- **Transformations:** if $Y=g(X)$, derive its distribution; e.g., standardizing $Y=(X-\mu)/\sigma$.

5. Algorithm / Working Process

Define what quantity is random, decide discrete or continuous, specify/estimate its PMF/PDF/CDF, then compute probabilities, summaries, or samples.

6. Mathematical Foundation

For discrete X , $\sum_x p(x)=1$. For continuous X , $P(a \leq X \leq b) = \int_a^b f(x) dx$ and $P(X=x)=0$. A valid CDF satisfies $F(-\infty)=0$, $F(\infty)=1$.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(4)
X = rng.poisson(lam=3, size=50_000) # RV: number of arrivals
print("P(X=3):", np.mean(X == 3))
print("P(X<=3):", np.mean(X <= 3))
```

8. Code Explanation

Each Poisson draw is one realization of RV X . Equality estimates a PMF value; the cumulative condition estimates a CDF value.

9. Training / Evaluation

Fit a distribution only after inspecting feature type, missing values, and outliers. Compare empirical and fitted CDFs/histograms; use held-out log likelihood where a parametric model is learned.

10. Complexity and Cost

Sampling/counting n values is $O(n)$. Multivariate RVs with many dimensions require many parameters or samples.

11. Common Use Cases

Sensor measurements, count labels, random augmentations, stochastic-gradient noise, RL rewards, and latent variables in VAEs.

12. Common Mistakes

Calling every column an RV without defining its population; assigning nonzero point probability to a continuous value; confusing a PDF height with a probability; ignoring units under transformations.

13. Edge Cases / Limitations

Real data may be mixed discrete-continuous, censored, skewed, or nonstationary. A scalar RV cannot express dependencies; use a random vector.

14. Variations

- **Random vector** contains several RVs; crucial for feature vectors.
- **Indicator RV** is 1 if an event occurs; simplifies probability derivations.
- **Latent RV** is unobserved; used in mixture models, VAEs, and HMMs.

15. Related Topics

Distributions describe RVs; expectation and variance summarize them; covariance measures joint random-vector behavior; probability models define likelihoods.

16. Interview Questions

1. **What is an RV?** A function from outcomes to numerical values.
2. **Discrete versus continuous?** Countable PMF versus interval-valued PDF.
3. **Can a continuous RV equal exactly 2 with positive probability?** Normally no.
4. **PDF versus CDF?** PDF is density; CDF is cumulative probability.
5. **How get interval probability?** Integrate PDF or subtract CDF values.
6. **Is PDF allowed above 1?** Yes; its area, not height, must integrate to 1.
7. **What is an indicator RV?** 1_A , equal to 1 when event A occurs.
8. **Why use RVs in ML?** They formalize uncertain features, labels, and predictions.
9. **What is a random vector?** An ordered collection of RVs.
10. **What is a realization?** One observed value/sample of an RV.

17. Practice Tasks

Create PMF/CDF from survey counts; simulate a Poisson arrival process; transform a normal RV; write a histogram-based empirical distribution.

18. Project Ideas

- **Queue simulator** — arrivals as Poisson RVs; NumPy; synthetic events; operations analytics.
- **Sensor-noise profiler** — model measurement distributions; Pandas/SciPy; UCI sensor data; data-quality portfolio piece.
- **VAE latent explorer** — visualize latent RV samples; PyTorch; MNIST; generative-AI foundation.

19. Quick Revision

An RV maps randomness to numbers. Discrete uses PMF; continuous uses PDF/CDF. Trap: PDF is not a point probability. One-liner: an RV is the mathematical object behind a random ML feature or label.

20. Final Cheat Sheet

Item	Summary
Input/output	experiment $\rightarrow$ numerical realization
Key rules	PMF sums to 1; PDF integrates to 1
Useful summaries	CDF, mean, variance
Best uses	data/noise/latent-variable modeling

Expectation

1. Overview

Expectation (mean) is the probability-weighted average of a random variable. It summarizes typical value and appears in expected loss, risk minimization, rewards, and gradient estimates.

2. Intuition

The expected die roll is 3.5, although 3.5 never appears. It is the average result over many identical experiments.

3. Prerequisites

Random variables, PMF/PDF, summation, integration intuition, and NumPy mean.

4. Core Concepts

- **Mean:** $E[X]$; center of a distribution, not necessarily a possible sample.
- **Linearity:** $E[aX+bY+c]=aE[X]+bE[Y]+c$, even if X, Y are dependent; common interview favorite.
- **Function expectation:** $E[g(X)]$ is found by weighting $g(x)$, not generally by $g(E[X])$.
- **Law of total expectation:** $E[X]=E[E[X|Y]]$; useful with groups and latent variables.

5. Algorithm / Working Process

For a known discrete distribution, multiply each value by its probability and sum. For data, calculate sample mean. For continuous distributions, integrate value times density.

6. Mathematical Foundation

$E[X]=\sum_x x p(x)$ (discrete), $E[X]=\int x f(x)dx$ (continuous), and sample estimate $\bar{x}=(1/n)\sum_i x_i$. Expected ML risk is $E_{(x,y)}[L(f(x),y)]$.

7. Practical Implementation

```
import numpy as np
faces = np.arange(1, 7)
probs = np.full(6, 1/6)
print("theoretical:", np.dot(faces, probs))
print("sample:", np.random.default_rng(1).choice(faces, 100_000).mean())
```

8. Code Explanation

`np.dot(values, probabilities)` implements the discrete expectation formula. The sampled mean converges to the theoretical mean as trials increase.

9. Training / Evaluation

Empirical risk minimization replaces unknown expected loss by mean training loss. Hold out validation data because a low training expectation can overfit. Report mean with dispersion, particularly on small test sets.

10. Complexity and Cost

A sample mean costs $O(n)$ time and $O(1)$ extra memory. Mini-batches estimate gradients cheaply but add variance; GPUs accelerate batch operations.

11. Common Use Cases

Average loss, expected return in RL, mean pooling, expected calibration error, imputation, and Monte Carlo inference.

12. Common Mistakes

Using mean for heavily skewed data without median; assuming $E[g(X)] = g(E[X])$; averaging non-comparable units; leaking test samples into an average used for training decisions.

13. Edge Cases / Limitations

Some heavy-tailed distributions have undefined expectation. The mean is sensitive to outliers and can hide multimodal groups.

14. Variations

- **Conditional expectation** summarizes X within a condition; core in regression.
- **Empirical expectation** is a dataset average; core in SGD.
- **Monte Carlo expectation** averages samples when integration is impossible; essential in Bayesian and generative methods.

15. Related Topics

Variance measures spread around expectation; MLE maximizes average log likelihood; cross-entropy is expected negative log probability; RL optimizes expected return.

16. Interview Questions

1. **Define expectation.** Probability-weighted average.
2. **Expected fair die roll?** 3.5.
3. **Must expectation be observable?** No.
4. **State linearity.** $E[aX+bY] = aE[X] + bE[Y]$.
5. **Does linearity need independence?** No.
6. **Is $E[X^2] = (E[X])^2$?** No, unless variance is zero.
7. **Why is mean loss used?** It estimates population risk and scales independently of dataset size.
8. **What is conditional expectation?** Expected X given information about another variable.
9. **Monte Carlo estimate?** Mean of sampled function values.
10. **Mean versus median?** Mean is outlier-sensitive; median is robust.

17. Practice Tasks

Calculate expected payoff of games; implement Monte Carlo integration; compare mean/median under outliers; show mini-batch loss variance across batch sizes.

18. Project Ideas

- **Monte Carlo option/pricing demo** — expected simulated payoff; NumPy; synthetic market paths; simulation skills.
- **RL bandit evaluator** — expected reward estimates; Gymnasium; bandit environment; RL fundamentals.
- **Robust-summary dashboard** — compare mean/median/trimmed mean; Pandas; housing data; practical analytics.

19. Quick Revision

Expectation is long-run average. Formula $\sum p(x)$ or sample $\sum x_i/n$. Use for loss/reward summaries. Trap: nonlinear transforms do not commute with expectation. One-liner: ML minimizes expected loss.

20. Final Cheat Sheet

Item	Summary
Definition	Weighted average of an RV
Key property	linearity without independence
Cost	$O(n)$ for sample mean
Best uses	loss, reward, Monte Carlo

Variance

1. Overview

Variance quantifies spread around the mean. It measures noise, estimator uncertainty, prediction reliability, and feature scale in ML pipelines.

2. Intuition

Two classes can both average 70 marks, but one may cluster near 70 while another ranges from 20 to 100. Variance captures this difference.

3. Prerequisites

Expectation, squares, random variables, and basic algebra.

4. Core Concepts

- **Variance:** $\text{Var}(X) = E[(X - \mu)^2]$; square prevents deviations cancelling.
- **Standard deviation** $\sigma = \sqrt{\text{Var}(X)}$ returns to original units and is often easier to interpret.
- **Computational identity:** $\text{Var}(X) = E[X^2] - (E[X])^2$; efficient but can be numerically delicate.
- **Sample variance:** use $n-1$ denominator for an unbiased estimate of population variance; interview staple.

5. Algorithm / Working Process

Compute mean, subtract it from each observation, square deviations, and average. For sample variance estimating a population, divide the sum by $n-1$.

6. Mathematical Foundation

$\text{Var}(aX+b)=a^2\text{Var}(X)$. For independent variables, $\text{Var}(X+Y)=\text{Var}(X)+\text{Var}(Y)$. Generally, $\text{Var}(X+Y)=\text{Var}(X)+\text{Var}(Y)+2\text{Cov}(X,Y)$.

7. Practical Implementation

```
import numpy as np
x = np.array([65, 68, 70, 72, 75], dtype=float)
print("population variance:", np.var(x))
print("sample variance:", np.var(x, ddof=1))
print("standard deviation:", np.std(x, ddof=1))
```

8. Code Explanation

`ddof=1` changes the divisor from n to $n-1$. Use it when the array is a sample used to estimate an unknown population variance.

9. Training / Evaluation

Standardize features using training-set mean/std only, then apply those statistics to validation/test. Monitor variance in losses/gradients across batches; use repeated runs and confidence intervals for model comparisons.

10. Complexity and Cost

Batch computation is $O(n)$ time, $O(1)$ extra memory. Online algorithms (Welford's method) update variance safely in $O(1)$ per point.

11. Common Use Cases

Feature scaling, PCA, anomaly detection, batch normalization statistics, uncertainty estimates, portfolio/risk models, and experiment variability.

12. Common Mistakes

Mixing sample and population formulas; fitting scalers before train/test split; interpreting variance in original units; ignoring outliers; assuming low variance always means a good model.

13. Edge Cases / Limitations

Variance is very outlier-sensitive and undefined for some heavy-tailed distributions. It cannot characterize asymmetric or multimodal uncertainty by itself.

14. Variations

- **Sample variance ($n-1$)** for unbiased estimation; placement-important.
- **Explained variance** in PCA/regression measures variability captured.
- **Regularized/pooled variance** stabilizes small groups; useful in statistics.
- **Online variance** supports data streams; useful in MLOps.

15. Related Topics

Standard deviation is square-root variance; covariance generalizes variance to feature pairs; Gaussian distributions are parameterized by mean and variance; bias-variance trade-off guides generalization.

16. Interview Questions

1. **Define variance.** Expected squared deviation from mean.
2. **Why square deviations?** To avoid cancellation and penalize large deviations.
3. **Variance versus standard deviation?** Squared units versus original units.
4. **Why $n-1$?** Bessel's correction makes sample variance unbiased under IID sampling.
5. **Can variance be negative?** No.
6. **$\text{Var}(aX+b)$?** $a^2\text{Var}(X)$.
7. **When do variances add?** For independent/uncorrelated variables (with zero covariance).
8. **How does scaling affect ML?** It affects distance, gradient, and regularization behavior.
9. **What does high bias/low variance mean?** Underfitting tendency.
10. **Robust alternative?** Median absolute deviation or IQR.

17. Practice Tasks

Implement variance without NumPy; compare **ddof** choices; standardize train/test safely; measure variance of cross-validation scores; implement Welford's algorithm.

18. Project Ideas

- **Anomaly detector** — flag high z-score points; Pandas/scikit-learn; credit-card data; statistical baseline.
- **Feature-scaling study** — compare KNN/SVM before/after scaling; scikit-learn; Wine data; interview-ready experiment.
- **Experiment stability tracker** — aggregate ML runs and uncertainty; MLflow/Pandas; own training logs; MLOps value.

19. Quick Revision

Variance is average squared spread: $E[(X-\mu)^2]$. Use for scale/noise. Trap: fit normalization on all data. One-liner: variance tells how unstable values are around their average.

20. Final Cheat Sheet

Item	Summary
Formula	$E[X^2] - E[X]^2$
Input/output	values $\rightarrow$ squared spread
Important choice	population n vs sample $n-1$

Item	Summary
Best uses	scaling, PCA, uncertainty

Common Distributions

1. Overview

A probability distribution specifies how likely each random-variable value is. Choosing an appropriate distribution lets ML systems model data, generate simulations, construct likelihoods, and quantify uncertainty.

2. Intuition

A distribution is a “shape of randomness”: coin outcomes, customer arrivals, measurement noise, and class probabilities all have different shapes and constraints.

3. Prerequisites

Probability basics, random variables, expectation, and variance.

4. Core Concepts

- **Discrete distributions** allocate probability mass to values (Bernoulli, Binomial, Poisson, Categorical).
- **Continuous distributions** allocate density over intervals (Gaussian, Uniform, Exponential).
- **Parameters** control shape, e.g., Gaussian μ, σ^2 ; estimating them is a learning task.
- **Support** is allowed values; choosing incompatible support is a modeling error.

5. Algorithm / Working Process

Inspect variable type and support, visualize histogram/ECDF, select candidate distributions from domain assumptions, estimate parameters, and validate fit with held-out likelihood or diagnostic plots.

6. Mathematical Foundation

A PMF sums to one; a PDF integrates to one. The expected log likelihood $(1/n) \sum \log p(x_i|\theta)$ is a standard distribution-fit score. CDFs allow probability queries and sampling by inverse transform.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(2)
bernoulli = rng.binomial(1, 0.3, size=10_000)
poisson = rng.poisson(4, size=10_000)
gaussian = rng.normal(10, 2, size=10_000)
print(bernoulli.mean(), poisson.mean(), gaussian.mean())
```

8. Code Explanation

NumPy generators sample common distributions. Empirical means should approach their theoretical parameters (p , λ , and μ) with enough samples.

9. Training / Evaluation

Fit on training samples; assess histogram/QQ plot, KS-style tests, and held-out log likelihood. Do not force a named distribution merely because it looks plausible; compare alternatives and downstream performance.

10. Complexity and Cost

Sampling most standard distributions is $O(n)$. Complex mixtures and multivariate densities add parameter and matrix costs; GPU is rarely necessary unless embedded in deep generative models.

11. Common Use Cases

Count forecasting, classifiers, noise injection, VAEs, mixture clustering, Bayesian priors, RL policies, and synthetic-data simulation.

12. Common Mistakes

Using normal distribution for bounded probabilities/counts; treating data as IID when it is temporal; fitting on test data; comparing PDFs by eye only; overlooking zero inflation and outliers.

13. Edge Cases / Limitations

Real data can be multimodal, correlated, censored, or shifted. A single simple distribution often underfits; a very flexible one may overfit.

14. Variations

- **Poisson** models nonnegative counts; use for arrivals when mean $\approx$ variance.
- **Exponential** models positive waiting times; use for memoryless processes.
- **Uniform** models equally likely bounded values; useful for simple priors.
- **Mixture distributions** combine components; use for clusters and multimodality.

15. Related Topics

Bernoulli/Binomial/Multinomial model categorical events; Gaussian models continuous noise; MLE fits parameters; multivariate Gaussian adds correlated features.

16. Interview Questions

1. **What is a distribution?** A rule assigning probabilities/densities to an RV's values.
2. **PMF versus PDF?** Mass at discrete values versus continuous density.
3. **Why support matters?** Model cannot validly assign behavior outside its allowed values.
4. **What is a parameter?** A value controlling distribution shape/location/scale.
5. **How select one?** Variable type, domain mechanism, plots, and held-out likelihood.
6. **Can normal model counts?** It is approximate only for large counts; Poisson/negative binomial are more natural.
7. **What is a mixture?** Weighted combination of distributions.

8. **Why use log likelihood?** It scores fit and makes products numerically manageable.
9. **IID assumption?** Samples are independent and identically distributed.
10. **What indicates mismatch?** Poor residual/QQ diagnostics or low held-out likelihood.

17. Practice Tasks

Fit Gaussian/Poisson candidates to a dataset; make histogram and ECDF; simulate a mixture; compare log likelihood on train/validation splits.

18. Project Ideas

- **Demand distribution fitter** — evaluate Poisson vs negative binomial; SciPy/Pandas; retail counts; forecasting relevance.
- **Synthetic-data generator** — fit and sample feature distributions; NumPy/SciPy; UCI data; data-augmentation understanding.
- **Distribution diagnostics app** — interactive histograms and QQ plots; Streamlit; any CSV; portfolio visualization.

19. Quick Revision

Distribution = full uncertainty model, constrained by support. Choose based on data-generating process. Trap: normalizing an invalid PMF/PDF incorrectly. One-liner: distributions turn raw uncertainty into a usable model.

20. Final Cheat Sheet

Item	Summary
Discrete	Bernoulli, Binomial, Poisson, Categorical
Continuous	Gaussian, Uniform, Exponential
Fit metric	held-out log likelihood
Best uses	likelihood, simulation, generative modeling

Gaussian Distribution

1. Overview

The Gaussian (normal) distribution is a bell-shaped continuous distribution governed by mean and variance. It models measurement noise and is foundational to regression, Kalman filters, Gaussian processes, and latent-variable models.

2. Intuition

Most values cluster near the mean; values far away are progressively rarer. Human measurement errors often arise from many small independent effects, making a Gaussian useful via the central limit theorem.

3. Prerequisites

Continuous RVs, expectation, variance, exponentials, and basic calculus intuition.

4. Core Concepts

- **Mean** μ locates the bell; **variance** σ^2 controls its width.
- **Standard normal** $N(0, 1)$ enables z-scores: $z = (x - \mu) / \sigma$.
- **68–95–99.7 rule**: roughly these percentages lie within 1, 2, 3 standard deviations.
- **Central limit theorem (CLT)**: sums/means of many suitable IID variables become approximately normal; interview favorite, not a claim that raw data is always normal.

5. Algorithm / Working Process

Estimate μ and σ^2 from training data, evaluate density or z-scores for new values, then use likelihood for fitting, anomaly scoring, or probabilistic predictions.

6. Mathematical Foundation

$p(x) = 1 / (\sqrt{2\pi\sigma^2}) \exp(-(x - \mu)^2 / (2\sigma^2))$. Negative log likelihood for independent data is proportional to squared error when variance is fixed—why linear regression uses MSE under Gaussian-noise assumptions.

7. Practical Implementation

```
import numpy as np
x_train = np.array([8.9, 10.1, 9.7, 10.5, 11.0])
mu, sigma = x_train.mean(), x_train.std(ddof=1)
x_new = 14.0
z = (x_new - mu) / sigma
print(mu, sigma, "z-score:", z, "anomaly?", abs(z) > 3)
```

8. Code Explanation

Mean and sample standard deviation estimate Gaussian parameters. A z-score measures distance from the estimated center in standard-deviation units.

9. Training / Evaluation

Fit parameters using training data. Validate normality using QQ plots/residual plots and evaluate held-out log likelihood. For Gaussian regression, inspect residuals, MSE/MAE, and predictive interval calibration.

10. Complexity and Cost

Univariate parameter fit is $O(n)$; density scoring is $O(1)$ per point. Multivariate Gaussian requires covariance inversion/factorization, typically $O(d^3)$.

11. Common Use Cases

Regression noise, anomaly detection, Kalman filtering, Gaussian mixture models, PCA assumptions, process control, and VAE latent priors.

12. Common Mistakes

Assuming bell-shaped histogram proves normality; applying z-scores to strongly skewed data; ignoring outliers; using zero/near-zero variance; confusing standard deviation with standard error.

13. Edge Cases / Limitations

Gaussian has unbounded support, so it may predict impossible negatives for positive-only quantities. It is poor for severe skew, multiple modes, and heavy tails.

14. Variations

- **Standard Gaussian** simplifies calculation; placement-essential.
- **Truncated Gaussian** respects bounds; useful for constrained measurements.
- **Gaussian mixture model** represents multiple clusters; projects/research important.
- **Gaussian process** places a Gaussian distribution over functions; advanced ML.

15. Related Topics

Multivariate Gaussian models correlated vectors; covariance controls its shape; MLE estimates μ, σ^2 ; linear regression's MSE follows Gaussian noise assumptions.

16. Interview Questions

1. **Parameters of a Gaussian?** Mean μ and variance σ^2 .
2. **What does mean change?** Location.
3. **What does variance change?** Spread.
4. **Standardize how?** $(x-\mu)/\sigma$.
5. **Why does MSE appear in regression?** It is Gaussian negative log likelihood with fixed variance.
6. **State 68–95–99.7 rule.** Approximate mass within 1,2,3 σ .
7. **Does CLT mean all data is Gaussian?** No; it concerns sums/means under conditions.
8. **Why is Gaussian convenient?** Closed-form properties and stable algorithms.
9. **When is it a poor model?** Bounded, skewed, heavy-tailed, or multimodal data.
10. **How detect anomalies?** Low density or large absolute z-score, after validating assumptions.

17. Practice Tasks

Generate Gaussian samples; fit parameters; create a QQ plot; compare MSE versus Gaussian NLL; test z-score anomaly detection under outliers.

18. Project Ideas

- **Sensor anomaly monitor** — Gaussian residual thresholding; Pandas/Streamlit; industrial sensors; practical monitoring.
- **Gaussian-noise regression** — estimate predictive intervals; scikit-learn; California Housing; uncertainty-aware regression.
- **GMM image segmenter** — color clustering; scikit-learn/OpenCV; natural images; CV portfolio strength.

19. Quick Revision

$N(\mu, \sigma^2)$ is a bell curve. Main density has squared distance divided by variance. Use for continuous symmetric noise. Trap: use it blindly for skew/bounds. One-liner: Gaussian converts distance from mean into probability density.

20. Final Cheat Sheet

Item	Summary
Input/output	$\mu, \sigma^2, x \rightarrow$ density/score/sample
Loss connection	Gaussian NLL $\approx$ MSE plus constants
Cost	$O(n)$ univariate fit
Pros/cons	tractable / sensitive to non-normality

Bernoulli Distribution

1. Overview

Bernoulli models one binary trial: success (1) or failure (0) with success probability p . It underlies binary labels, logistic regression, clicks, and yes/no experiments.

2. Intuition

One email is spam or not; one ad is clicked or not. The uncertain result is one bit, with a chance p of being 1.

3. Prerequisites

Probability basics, binary variables, expectation, and logarithms for likelihood.

4. Core Concepts

- **Support** $\{0, 1\}$; $P(X=1)=p$, $P(X=0)=1-p$.
- **Mean and variance**: $E[X]=p$, $\text{Var}(X)=p(1-p)$; maximum uncertainty occurs at $p=0.5$.
- **Indicator variable**: 1_A is Bernoulli with parameter $P(A)$.
- **Binary cross-entropy** is Bernoulli negative log likelihood; frequent interview connection.

5. Algorithm / Working Process

For observed labels, count successes; estimate p as success fraction. For classification, model $p(x)$ (usually sigmoid output), train by minimizing binary cross-entropy, and threshold or rank predictions at inference.

6. Mathematical Foundation

PMF: $P(X=x)=p^x(1-p)^{(1-x)}$. For labels y_i , NLL is $-\sum[y_i \log p_i + (1-y_i)\log(1-p_i)]$. MLE for IID observations is $\hat{p}=\sum x_i/n$.

7. Practical Implementation

```
import numpy as np
y = np.array([1, 0, 1, 1, 0, 1])
p_hat = y.mean()
eps = 1e-12
nll = -(y*np.log(p_hat + eps) + (1-y)*np.log(1-p_hat + eps)).mean()
print("MLE p:", p_hat, "binary NLL:", nll)
```

8. Code Explanation

The sample mean estimates success probability. The NLL scores how well one constant Bernoulli probability explains each binary target; `eps` avoids $\log(0)$.

9. Training / Evaluation

Use stratified train/validation/test splits if class imbalance exists. Train with binary cross-entropy; evaluate ROC-AUC/PR-AUC, precision, recall, F1, and calibration. Choose threshold from validation decision costs, not arbitrary 0.5.

10. Complexity and Cost

Constant-probability fitting is $O(n)$. A logistic model costs roughly $O(nd)$ per gradient pass. It is CPU-friendly; deep binary classifiers may use GPUs.

11. Common Use Cases

Spam detection, conversion prediction, disease presence, churn, binary segmentation pixels, and reward success events.

12. Common Mistakes

Using MSE rather than BCE without reason; reporting accuracy only on imbalanced labels; thresholding probabilities without calibration; forgetting numerical clipping; encoding positive class inconsistently.

13. Edge Cases / Limitations

It handles exactly one binary outcome, not counts or mutually exclusive multi-class labels. IID assumption can fail for repeated users/time series; extreme class imbalance makes raw p estimates unstable.

14. Variations

- **Logistic regression** models Bernoulli $p(x)$; core placement topic.
- **Beta-Bernoulli** adds a prior; useful for small-data rates/A-B tests.
- **Weighted/focal BCE** addresses imbalance; projects/DL important.

15. Related Topics

Binomial sums independent Bernoulli trials; categorical/multinomial generalize beyond two classes; sigmoid maps logits to Bernoulli parameters; MLE estimates p .

16. Interview Questions

1. **Bernoulli support?** $\{0, 1\}$.
2. **Parameter?** Success probability p .
3. **Mean/variance?** p and $p(1-p)$.
4. **Bernoulli versus Binomial?** One trial versus number of successes in n trials.
5. **Why BCE?** It is Bernoulli NLL.
6. **MLE of p ?** Sample mean.
7. **Why clip probabilities?** Avoid log zero and infinite loss.
8. **Best metric for rare positive class?** Often PR-AUC, recall/precision, and cost-sensitive metrics.
9. **What produces p in logistic regression?** Sigmoid of a linear logit.
10. **When is 0.5 threshold wrong?** Unequal class/costs or uncalibrated scores.

17. Practice Tasks

Derive MLE of p ; implement BCE from scratch; train logistic regression on imbalanced data; tune threshold for recall; add beta-prior smoothing.

18. Project Ideas

- **Churn predictor** — Bernoulli target with calibrated threshold; scikit-learn; Telco Churn; business ML.
- **CTR estimator** — beta-Bernoulli campaign rates; Pandas/Streamlit; ad logs/synthetic data; experimentation value.
- **Defect detector** — binary image classifier; PyTorch; NEU surface defects; DL portfolio.

19. Quick Revision

One yes/no trial: $P(x) = p^x(1-p)^{(1-x)}$. Mean p ; loss BCE. Trap: accuracy on imbalance. One-liner: Bernoulli is the probability model behind binary classification labels.

20. Final Cheat Sheet

Item	Summary
Input/output	$p \rightarrow$ one 0/1 outcome
Main formula	$E[X]=p, \text{Var}=p(1-p)$
Training loss	binary cross-entropy
Best uses	binary outcomes/classification

Binomial Distribution

1. Overview

Binomial models the number of successes in n independent Bernoulli trials with constant success probability p . It is useful for counts such as purchases among visitors or defects in a batch.

2. Intuition

Instead of modeling whether one visitor clicks, model how many of 100 similar independent visitors click. The count can range from 0 to 100.

3. Prerequisites

Bernoulli distribution, combinations, expectation, and variance.

4. Core Concepts

- **Parameters:** n trials and success probability p .
- **PMF:** combination counts arrangements of k successes; crucial interview derivation.
- **Mean/variance:** np and $np(1-p)$.
- **Assumptions:** fixed n , binary outcomes, independent trials, identical p ; audit these before use.

5. Algorithm / Working Process

Define a fixed trial window, count successes, estimate p (if unknown), then use Binomial PMF/CDF for probability, intervals, or hypothesis tests. For classification aggregates, sum Bernoulli predictions only when independence is defensible.

6. Mathematical Foundation

$P(X=k)=C(n,k)p^k(1-p)^{(n-k)}$, $k=0,\dots,n$. It is the distribution of $X=\sum_i B_i$ for IID $B_i \sim \text{Bernoulli}(p)$. For large n away from extreme p , normal approximation uses $N(np, np(1-p))$.

7. Practical Implementation

```
import numpy as np
from math import comb
n, p, k = 20, 0.3, 7
pmf = comb(n, k) * p**k * (1-p)**(n-k)
samples = np.random.default_rng(3).binomial(n, p, size=100_000)
print("P(X=7):", pmf, "empirical:", np.mean(samples == k))
```

8. Code Explanation

`comb(n,k)` counts sequences with k successes. NumPy simulation validates the theoretical PMF by observed frequency.

9. Training / Evaluation

Estimate p from historical representative trials and validate predicted count intervals on future windows. Check overdispersion: if observed variance exceeds $np(1-p)$, trials may be correlated or p may vary; use beta-binomial/negative-binomial alternatives.

10. Complexity and Cost

Computing one PMF value is $O(1)$ conceptually; evaluating all $n+1$ values is $O(n)$. Fit is $O(n)$ observed trials. No GPU required.

11. Common Use Cases

Batch quality control, email campaign conversions, clinical response counts, election polling, and A/B-test success totals.

12. Common Mistakes

Using Binomial for variable trial count; assuming independence for user/session events; confusing n with observed successes; using normal approximation for small/extreme counts; ignoring overdispersion.

13. Edge Cases / Limitations

It cannot handle unequal probabilities, dependence, more than two outcomes, or unbounded counts. At $p=0$ or 1 , variance is zero.

14. Variations

- **Poisson-binomial** permits different p_i ; useful for heterogeneous users.
- **Beta-binomial** makes p random; handles overdispersion.
- **Normal/Poisson approximations** speed calculations; placement-relevant rules of thumb.

15. Related Topics

Bernoulli is one trial; multinomial handles more than two classes; MLE estimates p ; confidence intervals and hypothesis tests commonly use Binomial counts.

16. Interview Questions

1. **What does Binomial model?** Number of successes in fixed IID Bernoulli trials.
2. **Parameters?** n, p .
3. **PMF?** $C(n, k)p^k(1-p)^{n-k}$.
4. **Mean/variance?** $np, np(1-p)$.
5. **Why combination term?** Successes can occur in any k of n positions.
6. **Bernoulli relation?** $\text{Binomial}(1, p)$ equals Bernoulli(p).
7. **Key assumptions?** Fixed trials, binary, independent, constant p .
8. **What is overdispersion?** Variance larger than model expectation.
9. **When normal approximation?** Large enough np and $n(1-p)$.
10. **Alternative for varying p ?** Poisson-binomial or beta-binomial.

17. Practice Tasks

Implement PMF without libraries; simulate confidence intervals; test normal approximation quality; model campaign conversions and diagnose overdispersion.

18. Project Ideas

- **Campaign forecast** — predict conversion-count intervals; Python/SciPy; marketing logs; business prediction.
- **Factory quality monitor** — detect abnormal defect counts; Pandas/Streamlit; synthetic/manufacturing data; operations focus.
- **Poll uncertainty visualizer** — Binomial sampling simulations; NumPy/Plotly; survey scenarios; clear statistical communication.

19. Quick Revision

Count successes from n IID binary trials. $E=np$, $Var=np(1-p)$. Trap: independent constant-rate assumption. One-liner: Binomial aggregates Bernoulli events into a success count.

20. Final Cheat Sheet

Item	Summary
Input/output	$n, p \rightarrow \text{count } 0 \dots n$
PMF	$C(n, k)p^k(1-p)^{(n-k)}$
Training	MLE $\hat{p}=\text{successes}/n$
Best uses	fixed-size binary batches

Multinomial Distribution

1. Overview

Multinomial models category counts across n independent trials, generalizing Binomial from two classes to K classes. It is central to class counts, bag-of-words, topic models, and categorical policy outcomes.

2. Intuition

Roll a six-sided die 100 times and record how many 1s, 2s, ..., 6s occur. The six counts jointly sum to 100; they are not independent because increasing one count leaves fewer trials for others.

3. Prerequisites

Categorical/Bernoulli concepts, Binomial distribution, combinations, vectors, and softmax.

4. Core Concepts

- **Parameters:** n and category vector $p=(p_1, \dots, p_K)$, with $\sum p_j=1$.
- **Count vector** $x=(x_1, \dots, x_K)$ satisfies $\sum x_j=n$.
- **PMF** uses multinomial coefficient $n!/(n x_j!)$.
- **Dependence:** category counts have negative covariance $-np_i p_j$; interview angle often missed.

5. Algorithm / Working Process

Define classes/probabilities, collect n independent categorical outcomes, form category counts, estimate $p_j = x_j/n$, and score or sample count vectors. In classifiers, softmax predicts p per observation; cross-entropy trains it.

6. Mathematical Foundation

$P(X=x) = n! / (\prod x_j!) \prod p_j^{x_j}$. $E[X_j] = np_j$, $\text{Var}(X_j) = np_j(1-p_j)$, $\text{Cov}(X_i, X_j) = -np_i p_j$ for $i \neq j$. NLL becomes multi-class cross-entropy $-\sum_j x_j \log p_j$.

7. Practical Implementation

```
import numpy as np
rng = np.random.default_rng(8)
p = np.array([0.2, 0.5, 0.3])
counts = rng.multinomial(n=20, pvals=p, size=5)
print(counts)
print("row sums:", counts.sum(axis=1))
print("MLE p from first row:", counts[0] / 20)
```

8. Code Explanation

Each row is one count vector for 20 categorical trials. Dividing category counts by total trials gives the MLE probability vector.

9. Training / Evaluation

For classification, use stratified splits, softmax cross-entropy, confusion matrix, macro/micro F1, per-class recall, top-k accuracy, and calibration. Handle imbalanced categories with weights, resampling, or focal loss; do not duplicate test data.

10. Complexity and Cost

Counting is $O(n)$; storing probabilities is $O(K)$. Deep softmax costs $O(K)$ per prediction and is expensive for huge vocabularies; sampled/hierarchical softmax can help.

11. Common Use Cases

Document word counts, multi-class image labels, dice/category experiments, topic models, language-model vocabulary probabilities, and RL discrete actions.

12. Common Mistakes

Using independent Binomials for mutually exclusive counts; allowing probabilities not to sum to one; applying sigmoid instead of softmax for single-label multi-class tasks; ignoring class imbalance.

13. Edge Cases / Limitations

Requires fixed n , independent trials, and constant category probabilities. Sparse high-cardinality vocabularies are costly; zero observed category counts make unsmoothed log likelihood infinite on future events.

14. Variations

- **Categorical** is one trial ($n=1$); essential for softmax classifiers.
- **Dirichlet-multinomial** adds uncertainty in p ; used in topic modeling/smoothing.
- **Multilabel Bernoulli** uses independent binary labels instead; use when labels can co-occur.
- **Hierarchical/sampled softmax** scales huge vocabularies; relevant to LLMs.

15. Related Topics

Binomial is $K=2$; categorical is $n=1$; softmax outputs a categorical distribution; cross-entropy is multinomial NLL; Dirichlet is a prior over p .

16. Interview Questions

1. **What does multinomial model?** Counts over K categories in n categorical trials.
2. **Parameter constraint?** $p_j \geq 0$ and $\sum p_j = 1$.
3. **How differs from multivariate normal?** Multinomial is discrete counts, not continuous vectors.
4. **Binomial relation?** Two-category multinomial equals Binomial.
5. **Expected category count?** np_j .
6. **Are category counts independent?** No; they must sum to n .
7. **Why softmax?** Produces valid category probabilities summing to one.
8. **Loss for one-label K-class task?** Categorical cross-entropy.
9. **What is categorical distribution?** One multinomial trial.
10. **How avoid zero probabilities?** Dirichlet/Laplace smoothing or learned regularized model.

17. Practice Tasks

Simulate die-count vectors; derive class-count covariance; implement stable softmax; compare softmax versus independent sigmoids on multi-class data; add Laplace smoothing to a word model.

18. Project Ideas

- **Text-category classifier** — word-count multinomial Naive Bayes; scikit-learn; 20 Newsgroups; NLP staple.
- **Image classifier audit** — evaluate class distribution/confusion; PyTorch; CIFAR-10; DL interviewing value.
- **Topic-count simulator** — Dirichlet-multinomial documents; NumPy; synthetic/public corpus; generative modeling foundation.

19. Quick Revision

Multinomial counts K mutually exclusive outcomes: $\sum x = n$. Use for multi-class/count data. Trap: categories are dependent. One-liner: it is Binomial with more than two categories.

20. Final Cheat Sheet

Item	Summary
Input/output	$n, p_1 \dots p_K \rightarrow$ category-count vector
Mean/covariance	μ_j ; off-diagonal σ_{ij}
Training loss	multi-class cross-entropy
Best uses	class/word counts, softmax systems

Maximum Likelihood Estimation

1. Overview

Maximum likelihood estimation (MLE) chooses model parameters that make observed training data most probable. It is the principle behind linear regression, logistic regression, Naive Bayes, Gaussian models, and neural-network cross-entropy training.

2. Intuition

Given several candidate coin biases, choose the bias under which the observed head/tail sequence would be most plausible. MLE “explains what was seen” without adding a prior preference.

3. Prerequisites

Probability distributions, independence, logarithms, derivatives/optimization basics, and loss functions.

4. Core Concepts

- **Likelihood** $L(\theta; D) = p(D|\theta)$ treats data as fixed and parameters as candidates.
- **Log likelihood** turns products into sums and improves numerical stability.
- **NLL loss** $-\log L$ is minimized in software; it connects probability models to optimization.
- **iid assumption** gives $p(D|\theta) = \prod_i p(x_i|\theta)$; recognize when this is questionable.

5. Algorithm / Working Process

1. Choose a probabilistic model $p(\text{data}|\theta)$.
2. Write likelihood over training samples.
3. Take logs; optionally add regularization.
4. Differentiate/optimize using closed form, gradient descent, or EM.
5. Evaluate held-out likelihood and task metrics; use learned parameters at inference.

6. Mathematical Foundation

$\hat{\theta}_{MLE} = \arg\max_{\theta} \prod_i p(x_i|\theta) = \arg\max_{\theta} \sum_i \log p(x_i|\theta)$. Gaussian known-variance MLE for mean is $\bar{x}$; Bernoulli MLE is sample mean. L2 regularization corresponds to Gaussian-prior MAP, not pure MLE.

7. Practical Implementation

```
import numpy as np
y = np.array([1, 1, 0, 1, 0, 1, 1])
grid = np.linspace(0.001, 0.999, 999)
log_likelihood = (y[:, None] * np.log(grid) +
                  (1 - y[:, None]) * np.log(1 - grid)).sum(axis=0)
p_mle = grid[np.argmax(log_likelihood)]
print("grid MLE:", p_mle, "closed form:", y.mean())
```

8. Code Explanation

Each grid value is a candidate Bernoulli parameter. Summing log probabilities produces its log likelihood; `argmax` selects the best. The analytic MLE is exactly the label mean.

9. Training / Evaluation

Use training data to optimize NLL and validation data for early stopping/model selection. Evaluate both NLL (probability quality) and task metrics (accuracy, F1, RMSE). Regularization, cross-validation, and data augmentation combat overfitting.

10. Complexity and Cost

Depends on model. Closed forms can be fast; iterative optimization costs approximately iterations $\times$ data $\times$ parameters. Neural-network MLE via SGD is GPU-intensive; log-sum-exp and minibatches improve numerical/computational behavior.

11. Common Use Cases

Linear/logistic regression, Gaussian parameter fit, HMMs, GMMs, language models, neural classifiers, and generative models.

12. Common Mistakes

Maximizing probability density while comparing incompatible units; multiplying tiny probabilities instead of logs; optimizing on test data; forgetting identifiability; confusing likelihood with posterior probability; ignoring regularization.

13. Edge Cases / Limitations

MLE can overfit with limited data, be biased in finite samples, have no finite optimum (e.g., separable logistic data), or be sensitive to outliers/model misspecification.

14. Variations

- **MAP** adds prior and regularizes estimates; placement-important comparison.
- **Maximum conditional likelihood** trains discriminative classifiers $p(y|x)$.
- **EM algorithm** maximizes/increases likelihood with latent variables; GMM/HMM relevant.
- **Penalized MLE** adds L1/L2 penalties; essential in practical models.

15. Related Topics

Cross-entropy is categorical/Bernoulli NLL; Gaussian MLE leads to sample mean/variance; Bayes adds a prior to form posterior; KL divergence links NLL to distribution matching.

16. Interview Questions

1. **Define MLE.** Parameters maximizing likelihood of observed data.
2. **Why log likelihood?** Products become sums and avoid underflow.
3. **MLE of Bernoulli p ?** Sample mean.
4. **MLE Gaussian mean?** Sample mean.
5. **MLE versus MAP?** MAP includes prior; MLE does not.
6. **Why can MLE overfit?** It has no inherent preference for simpler/plausible parameters.
7. **What does IID enable?** Factorization of data likelihood into per-sample terms.
8. **Cross-entropy connection?** It is negative log likelihood for categorical labels.
9. **Why use mini-batch SGD?** Cheaper noisy gradient estimate for large datasets.
10. **What is identifiability?** Different parameters should imply distinguishable distributions.

17. Practice Tasks

Derive Bernoulli/Gaussian MLEs; implement logistic NLL and gradient descent; compare MLE/MAP with small samples; fit a GMM using EM; plot likelihood landscapes.

18. Project Ideas

- **From-scratch logistic regression** — BCE/MLE with gradient descent; NumPy; Breast Cancer data; math-to-code proof.
- **GMM customer clustering** — EM likelihood fitting; scikit-learn; Mall Customers; unsupervised ML project.
- **Language-model NLL analyzer** — token likelihood/perplexity; PyTorch; WikiText subset; LLM metrics insight.

19. Quick Revision

MLE chooses θ that maximizes $p(\mathcal{D}|\theta)$, usually $\sum \log p$. Use for probabilistic parameter learning. Trap: likelihood is not $p(\theta|\mathcal{D})$. One-liner: most standard ML losses are negative log likelihoods in disguise.

20. Final Cheat Sheet

Item	Summary
Input/output	data + model family $\rightarrow$ fitted parameters
Objective	maximize log likelihood / minimize NLL
Optimization	closed form, GD, EM
Pros/cons	principled, efficient / can overfit or be misspecified

Covariance Matrix

1. Overview

A covariance matrix records variances of features and their pairwise co-variation. It is fundamental to PCA, multivariate Gaussian models, whitening, portfolio risk, feature analysis, and Mahalanobis distance.

2. Intuition

If height and weight tend to increase together, their covariance is positive. A covariance matrix places every feature's spread on the diagonal and each pair's joint movement off-diagonal.

3. Prerequisites

Vectors/matrices, expectation, variance, random vectors, and basic linear algebra.

4. Core Concepts

- **Covariance:** $\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$; sign indicates linear co-movement.
- **Matrix Σ :** $\Sigma_{ij} = \text{Cov}(X_i, X_j)$; diagonal entries are variances.
- **Correlation:** covariance normalized to $[-1, 1]$; use it for scale-free comparison.
- **Positive semidefinite (PSD):** $v^T \Sigma v \geq 0$; valid covariance matrices are symmetric PSD. Interview angle: eigenvalues cannot be negative.

5. Algorithm / Working Process

Arrange data as $n \times d$ matrix, center each training-set feature, compute centered transpose times centered data divided by $n-1$, inspect correlations/eigenvalues, and reuse training centering for new data.

6. Mathematical Foundation

Population: $\Sigma = E[(X - \mu)(X - \mu)^T]$. Sample: $S = (1/(n-1))X_c^T X_c$. Covariance transforms as $\text{Cov}(AX+b) = A\Sigma A^T$. PCA finds eigenvectors of S .

7. Practical Implementation

```
import numpy as np
X = np.array([[1, 2], [2, 3], [3, 5], [4, 4]], dtype=float)
cov = np.cov(X, rowvar=False, ddof=1)
correlation = np.corrcoef(X, rowvar=False)
print(cov)
print(correlation)
```

8. Code Explanation

Rows are examples and columns features, hence `rowvar=False`. `np.cov` centers variables internally; diagonal entries are sample variances.

9. Training / Evaluation

Fit covariance only on training data to prevent leakage. Check sample size relative to dimensions, condition number, eigenvalues, and downstream validation metrics. Standardize before PCA/correlation when units differ; use shrinkage with few samples.

10. Complexity and Cost

Computing dense covariance is $O(nd^2)$ time and $O(d^2)$ memory; eigendecomposition/inversion is $O(d^3)$. For high-dimensional embeddings, use randomized PCA, diagonal approximations, or low-rank methods; GPUs can help.

11. Common Use Cases

PCA, feature redundancy detection, Gaussian classifiers, whitening, finance risk, Mahalanobis anomaly detection, and embedding analysis.

12. Common Mistakes

Confusing covariance with correlation/causation; failing to center; leaking scaling/covariance statistics; inverting singular matrices; using covariance on nonlinear relationships; using n versus $n-1$ inconsistently.

13. Edge Cases / Limitations

Sample covariance is noisy when d is comparable to/exceeds n , becomes singular when $d \geq n$ after centering, and is outlier-sensitive. Zero covariance does not imply independence except in special families such as jointly Gaussian variables.

14. Variations

- **Correlation matrix** normalizes scales; useful for exploratory analysis.
- **Diagonal covariance** assumes features independent; fast but restrictive.
- **Shrinkage covariance** blends with structured target; excellent when samples are scarce.
- **Robust covariance** reduces outlier influence; useful in anomaly detection.

15. Related Topics

Variance is diagonal covariance; multivariate Gaussian needs a covariance matrix; PCA diagonalizes covariance; whitening transforms covariance near identity; attention/embeddings often benefit from covariance analysis.

16. Interview Questions

1. **What is covariance?** Expected product of centered variables.
2. **Diagonal of covariance matrix?** Feature variances.
3. **Why symmetric?** $\text{Cov}(X, Y) = \text{Cov}(Y, X)$.
4. **Covariance versus correlation?** Correlation is scale-normalized covariance.
5. **Does zero covariance mean independence?** Not generally.
6. **Why PSD?** Every projected variance $\mathbf{v}^T \Sigma \mathbf{v}$ must be nonnegative.
7. **Why center data?** Covariance measures deviations from means.

8. **Covariance transform under A?** $A\Sigma A^T$.
9. **Why singular covariance is problematic?** Cannot directly invert for Gaussian density/Mahalanobis distance.
10. **PCA connection?** Principal directions are covariance eigenvectors.

17. Practice Tasks

Calculate a 2D covariance manually; compare covariance/correlation under rescaling; run PCA; demonstrate singularity with duplicate features; compare empirical and shrinkage covariance.

18. Project Ideas

- **PCA face/image compression** — covariance eigenvectors; scikit-learn; Olivetti faces; classic linear algebra project.
- **Portfolio-risk explorer** — asset covariance and diversification; Pandas/Plotly; market returns; quantitative analytics.
- **Embedding redundancy audit** — covariance/PCA for vector features; NumPy; sentence embeddings; AI-engineering relevance.

19. Quick Revision

Covariance measures joint linear variation; matrix diagonal = variances. Formula $E[(X-\mu)(X-\mu)^T]$. Trap: covariance is scale-dependent and not causation. One-liner: covariance is the geometry of feature spread and co-movement.

20. Final Cheat Sheet

Item	Summary
Input/output	$n \times d$ data $\rightarrow d \times d$ symmetric PSD matrix
Cost	$O(nd^2)$ time, $O(d^2)$ memory
Key use	PCA, Gaussian densities, whitening
Limitation	noisy/singular in high dimensions

Multivariate Gaussian

1. Overview

The multivariate Gaussian (MVG) generalizes the Gaussian to a vector of correlated continuous features. It models joint feature density in Gaussian discriminant analysis, anomaly detection, Kalman filters, GMMs, and VAEs.

2. Intuition

In two dimensions, a Gaussian is an ellipse-shaped cloud rather than a bell curve. Mean sets the center; covariance determines width, rotation, and correlation of the ellipse.

3. Prerequisites

Gaussian distribution, covariance matrices, vectors/matrices, determinant, inverse, and basic linear algebra.

4. Core Concepts

- **Parameters:** mean vector $\mu \in \mathbb{R}^d$ and covariance matrix $\Sigma \in \mathbb{R}^{d \times d}$.
- **Contours:** equal-density points form ellipses/ellipsoids; covariance orientation captures correlation.
- **Mahalanobis distance** $(x-\mu)^T \Sigma^{-1} (x-\mu)$ measures distance accounting for scale/correlation.
- **Marginals/conditionals** of an MVG remain Gaussian; a powerful inference property.

5. Algorithm / Working Process

1. Collect continuous feature vectors and clean/impute consistently.
2. Fit μ and Σ on training/inlier data.
3. Stabilize Σ (regularization/shrinkage) and factorize it with Cholesky rather than explicit inverse.
4. Score new vectors by log density or Mahalanobis distance.
5. For supervised use, fit one Gaussian per class and compare class log posteriors.

6. Mathematical Foundation

$p(x) = 1 / ((2\pi)^{d/2} |\Sigma|^{1/2}) \exp[-\frac{1}{2}(x-\mu)^T \Sigma^{-1} (x-\mu)]$. MLE gives $\hat{\mu} = (1/n) \sum x_i$, $\hat{\Sigma} = (1/n) \sum (x_i - \hat{\mu})(x_i - \hat{\mu})^T$ (note MLE uses n ; unbiased covariance uses $n-1$). Log density avoids underflow.

7. Practical Implementation

```
import numpy as np
from scipy.stats import multivariate_normal

X_train = np.array([[0, 1], [1, 2], [2, 2], [2, 3], [3, 4]], dtype=float)
mu = X_train.mean(axis=0)
cov = np.cov(X_train, rowvar=False) + 1e-6 * np.eye(2) # jitter
x_new = np.array([5.0, 0.0])
log_density = multivariate_normal.logpdf(x_new, mean=mu, cov=cov)
mahal_sq = (x_new-mu) @ np.linalg.solve(cov, x_new-mu)
print("log density:", log_density, "squared Mahalanobis:", mahal_sq)
```

8. Code Explanation

The training mean/covariance define the MVG. Small diagonal **jitter** makes near-singular covariance invertible. **solve** is numerically preferable to forming **inv(cov)**; low log density/high Mahalanobis distance indicates an outlier.

9. Training / Evaluation

Split by time/entity where appropriate and fit only on training/inliers. Evaluate anomaly detection with PR-AUC, ROC-AUC, recall at fixed false-positive rate, and calibrated thresholds. For class models, use accuracy/F1 plus log likelihood. Standardize features and validate covariance regularization strength.

10. Complexity and Cost

Fitting covariance costs $O(nd^2)$; Cholesky factorization is $O(d^3)$; each full-covariance score is $O(d^2)$ after factorization. Memory is $O(d^2)$. High-dimensional models require diagonal/low-rank covariance, PCA, or shrinkage; GPUs are helpful only at scale.

11. Common Use Cases

Multivariate anomaly detection, Gaussian discriminant analysis (LDA/QDA), Kalman filters, GMM components, sensor fusion, face recognition embeddings, and correlated latent priors.

12. Common Mistakes

Using explicit matrix inverse; fitting an unregularized covariance with too few rows; omitting scaling; treating low density as automatically malicious; using Gaussian density on categorical data; confusing MLE and unbiased covariance denominators.

13. Edge Cases / Limitations

Full covariance is singular/unstable in high dimensions, sensitive to outliers, and poorly matched to non-elliptical/multimodal data. Density values shrink with dimension, so threshold log density/Mahalanobis score using validation—not intuition.

14. Variations

- **Diagonal MVG** assumes conditional feature independence; scalable and placement-relevant.
- **Spherical MVG** uses one shared variance; fastest but restrictive.
- **Gaussian mixture model** models multiple ellipses; use for clusters/multimodality.
- **LDA/QDA** use shared versus per-class covariance; core interview comparison.
- **Factor analysis/low-rank Gaussian** reduces covariance cost; research/high-dimensional use.

15. Related Topics

Covariance matrix supplies Σ ; univariate Gaussian is $d=1$; GMM combines MVGs; Mahalanobis distance defines covariance-aware anomaly scores; PCA reduces dimensions before MVG fitting.

16. Interview Questions

1. **Parameters of MVG?** Mean vector and covariance matrix.
2. **What determines ellipse orientation?** Off-diagonal covariance/eigenvectors.
3. **Why must covariance be PSD?** It represents valid nonnegative projected variances.
4. **MVG density formula key term?** Mahalanobis quadratic form.
5. **Why logpdf?** Avoid numerical underflow and turn products into sums.
6. **Why not invert covariance explicitly?** It is less numerically stable and slower than solving/factorization.
7. **What if covariance is singular?** Add regularization, reduce dimensions, or use diagonal/shrinkage covariance.
8. **LDA versus QDA?** LDA shares covariance across classes; QDA learns one per class.
9. **Are zero covariance features independent?** In a jointly Gaussian distribution, yes; not generally.

10. **How detect anomaly?** Threshold held-out calibrated low log density or large Mahalanobis distance.

17. Practice Tasks

Plot 2D covariance ellipses; derive MVG log likelihood; compare full/diagonal covariance anomaly detectors; add covariance shrinkage; implement LDA and QDA comparison.

18. Project Ideas

- **Multisensor anomaly detector** — MVG scores for equipment; Pandas/SciPy; sensor dataset; production-relevant monitoring.
- **LDA/QDA classifier benchmark** — compare covariance assumptions; scikit-learn; Wine/Breast Cancer; interview-grade analysis.
- **GMM customer segments** — multi-Gaussian clustering and visualizations; scikit-learn/Plotly; retail features; unsupervised portfolio project.

19. Quick Revision

MVG models correlated continuous vectors: $\mathcal{N}(\mu, \Sigma)$. Its key geometry is Mahalanobis distance. Use for elliptical joint data. Trap: full covariance needs enough clean training samples. One-liner: MVG is a Gaussian cloud whose covariance sets its shape and tilt.

20. Final Cheat Sheet

Item	Summary
Input/output	feature vector $\rightarrow$ joint density/anomaly score
Main parameters	mean vector μ , PSD covariance Σ
Cost	fit $\mathcal{O}(nd^2)$, factorize $\mathcal{O}(d^3)$
Pros/cons	models correlation / costly, sensitive to mismatch
Best uses	anomalies, GDA, GMM, sensor fusion